



THE HACKBOOK

HackTVM '26

Contents

1. Chapter 1: Core Directive - - - - -	1
1.1 The Theme	
1.2 The Purpose	
1.3 The Target	
2. Chapter 2: Standards & Inspiration- - - - -	4
2.1 Building It Right	
2.2 Major Industry Standards	
2.3 Case Studies	
2.4 Further Reading	
3. Chapter 3: Technical Stack & Tooling - - - -	19
3.1 Useful Hardware	
3.2 Accessibility Design Toolkits	
3.3 Specialized Accessibility APIs	
3.4 General Hackathon Toolkit	
4. Chapter 4: Protocols - - - - -	35
4.1 Rules and Guidelines	
4.2 Components and Tools	
4.3 AI and External Resources	
4.4 Originality and Fair Play	
4.5 Judging Rubric - Phase I	
4.6 Venue Details	

Chapter 1:

THE CORE DIRECTIVE

1.1 The Theme

Access Point challenges students to design working technology that closes real, everyday gaps in accessibility and inclusivity, whether that's in navigating a space, communicating, learning, working or getting basic public information everyone else takes for granted.

1.2 The Purpose

Why Does This Theme Matter?

Modern design frequently optimizes for an "idealized default user", treating edge cases as an afterthought. *Access Point* shifts this dynamic. Access to physical spaces, communication, learning, employment, and public information is a fundamental right, not an optional feature. Technology is one of the most direct levers for removing systemic exclusion and it must be leveraged to grant individuals autonomy over their daily lives.

The Technical Challenge

Building for accessibility and inclusivity forces you to confront some of the most demanding engineering problems in computer science and product design.

When you remove standard assumptions—like a touch screen, clear visual field, and predictable physical inputs—you can no longer rely on easy architectural shortcuts. Every system must be leaner, faster, and remarkably resilient, with zero room for error

- **Operating Under High Constraints:** Standard software can afford a few extra milliseconds of latency; an assistive communication device or real-time navigation system cannot. Building tools that parse environment data in real time demands optimized pipelines, lightweight models, and efficient resource management right at the edge.
- **Multimodal Data & Sensor Fusion:** Removing a barrier often means translating one physical sensory channel into another. This

requires bridging complex data streams—converting spatial visual depth into dynamic haptic patterns, or turning unstructured audio into clean visual interfaces—without losing fidelity or overwhelming the user.

- **Parsing Unstructured, Real-World Inputs:** Humans do not interact with technology in sterile laboratory conditions. Adaptive interfaces must handle noisy environments, non-standard motor inputs, variable lighting, and unpredictable physical terrain. Your algorithms must be resilient enough to handle messy, real-world edge cases reliably.
- **Zero-Tolerance Reliability:** Assistive tech lives on the critical path of a user's daily life. If a mainstream app crashes, it's a minor inconvenience; if an accessibility tool fails, it halts a person's ability to navigate, communicate, or work independently. The engineering standard required here is fundamentally higher because the stakes are real.

1.3 The Target

The core mandate of Access Point is rooted in a fundamental shift in perspective: you are building to remove the friction in the environment or system, not to “fix” or “cure” the human being.

- **Target the Barrier:** focus on what is actually stopping the user from being fully independent. Remove the friction that they face, but others don't
- **Do not medicalize the person:** Avoid projects aiming at repairing a disability or treating disabled people as passive subjects of pity
- **Design around specific needs rather than vague assumptions.**

Chapter 2:

STANDARDS & INSPIRATION

2.1 Building It Right

The following 6 questions will help you narrow down on where you need to aim before you start building your solutions

1. Where will your user actually use it?

Not on your laptop, on your Wi-Fi, in a quiet room. Think about worse conditions: one hand busy, a noisy street, a weak signal, a cheap phone, a dying battery. Design for that, not for the demo.

(ISO 9241-210: design around the real context of use.)

2. Can they use it without you in the room?

If it needs someone to set it up, explain it or hold it, it isn't independent yet. A first-time user should get the core function without a manual.

(WCAG 2.2 "Understandable.")

3. Did you choose your channels on purpose?

Every tool takes input somehow (touch, voice, gesture, a button) and gives output somehow (sound, vision, vibration, text). Pick these deliberately, based on your user, instead of defaulting to what's easiest to code. Don't assume a sense, a hand, or a connection speed your user may not have. *(WCAG 2.2 "Perceivable" and "Operable." EN 301 549 for hardware.)*

4. What happens when it breaks?

It will break: battery, signal, misheard input. Decide now what your user gets when that happens, even if the answer is just "a clear message and a simpler fallback."

(WCAG 2.2 "Robust.")

5. What does it cost the user?

Data, dignity, money and effort. Collect the minimum, especially anything from a camera or mic. Don't make the user stand out more than they already do. If it's hardware, keep it physically safe.

(WCAG 2.2, plus EN 301 549 on hardware safety.)

6. Has someone already solved this?

Search before you build. Build the gap, not the clone, or improve on what

exists. One core function done reliably beats five half-working ones. *(ISO 9999 classifies assistive products, which makes it a handy map of what's out there.)*

2.2 Major Industry Standards

These are the frameworks that accessibility work is measured against. You won't be certifying your project against them. They're useful because they compress decades of experience into a few principles you can build with.

1. The HAAT Model

A framework from assistive technology engineering and rehabilitation. It treats every solution as a system of four parts, and the design only works if all four fit together:

- **Human:** the person, with their abilities, skills and preferences. Design around what they *can* do.
- **Activity:** the specific thing they're trying to do. "Read a bus number" is an activity. "Help with travel" isn't.
- **Assistive Technology:** the technology being used to support the activity. Consider how the person interacts with it, what it does, how it provides feedback, and how it interacts with the surrounding environment.
- **Context:** where it happens, including physical, social, cultural and institutional factors. A device that works at home can fail in a crowded classroom or on a noisy street.

2. WCAG 2.2 (Web Content Accessibility Guidelines)

The global benchmark for web accessibility, published by the W3C and also adopted as ISO/IEC 40500. It applies to websites and web applications. Its principles are also widely used as guidance

when designing mobile and other digital interfaces. It is built on four principles, known as **POUR**:

- **Perceivable:** Users must be able to take in the information. This means text alternatives for images, captions for audio, content that adapts to different presentations (such as a screen reader or larger text), and enough contrast that nothing depends on colour alone.
- **Operable:** Users must be able to control the interface. Users should be able to operate functionality through a keyboard interface where applicable, rather than requiring a mouse or other pointing device. Touch and gesture-based interactions should also have accessible alternatives where required. Content should avoid flashes that could avoid seizures, have a minimum target size etc
- **Understandable:** Users must be able to follow what's happening. Language should be clear, behaviour should be consistent and predictable, and forms should identify errors and help users fix them.
- **Robust:** The content must work with a range of browsers, devices and assistive technologies, including ones that don't exist yet. In practice this means clean, correctly labelled structure that screen readers and similar tools can interpret.

WCAG has three conformance levels: **A** (the minimum), **AA** (the practical target most laws and organisations use) and **AAA** (the highest, not always achievable for all content).

3. EN 301 549: The Hardware-Friendly Standard

Europe's standard for accessible ICT. It incorporates WCAG requirements for web content and includes additional requirements covering software, hardware, documents, support services and other ICT. It borrows WCAG for software and adds requirements for hardware. Its best idea for you is that it starts from *how people use things*:

- Using it without vision, or with limited vision
- Using it without hearing, or with limited hearing
- Using it with limited strength, reach or dexterity
- Using it with limited cognition or learning
- Avoiding seizure triggers
- Protecting privacy

4. The 7 Principles of Universal Design

Developed at North Carolina State University's Center for Universal Design, these are the foundation for designing physical products and spaces so the widest range of people can use them:

1. **Equitable use:** the design is useful to people with diverse abilities.
2. **Flexibility in use:** it accommodates a range of preferences and abilities.
3. **Simple and intuitive use:** it's easy to understand regardless of experience or concentration level.
4. **Perceptible information:** it communicates necessary information effectively, whatever the user's sensory abilities.
5. **Tolerance for error:** it minimises the consequences of accidents and unintended actions.
6. **Low physical effort:** it can be used comfortably with minimal fatigue.
7. **Size and space for approach and use:** it provides appropriate size and space for reach and manipulation, whatever the user's body size, posture or mobility.

5. ISO 9241-210 (Human-Centred Design)

Where many accessibility standards describe requirements or principles for accessible products, this standard focuses on the *human-centred design process*. Its principles:

- The design is based on an explicit understanding of users, tasks and environments.
- Users are involved throughout design and development.
- The design is driven and refined by user-centred evaluation.
- The process is iterative.
- The design addresses the whole user experience, not just one interaction.
- The design team includes multidisciplinary skills and perspectives.

Also Worth Knowing

- **ISO 9999** provides a classification system for assistive products, making it useful when researching existing solutions and identifying gaps
- **India's Rights of Persons with Disabilities Act, 2016** establishes legal protections and accessibility obligations for persons with disabilities, including requirements relating to accessibility and reasonable accommodation.

2.3 Case Studies

1. WeWALK Smart Cane (wewalk.io)

The barrier:

A conventional white cane is primarily designed to detect obstacles and changes in the ground ahead for visually impaired people. It does not reliably detect obstacles at upper-body or head height.

What it is:

A smart handle that detects obstacles above chest level using an ultrasonic and other sensors, and warns the user with haptic and audible feedback. WeWALK began inside Young Guru Academy in Turkey, then spun off as its own company in 2018 and joined Microsoft's AI for Accessibility programme in 2020.

WeWALK Smart Cane 2 was introduced at CES 2025 with a redesign, more advanced sensors from TDK, and an AI voice assistant so users don't need to juggle a smartphone. It has four modes, from a plain classic cane that works even when not connected, up to navigation and AI features. In the higher modes, users can create navigation routes and ask for directions or nearby places from the cane itself.

Why it works (HAAT):

- Human: an experienced white cane user.
- Activity: travelling independently, hands free.
- Assistive technology: sensor, buttons, voice assistant, smartphone app and navigation software, all in a handle that swaps onto the cane they already trust.
- Context: crowded streets, luggage, noise.

Where it falls short:

- It must be used with a white cane and shouldn't be used without prior white-cane training.
- Cost. The Smart Cane 2 is currently listed by WeWALK at \$850. AI and other features are sold through different packages, with subscription requirements varying by region and selected add-ons.

Takeaway:

Find the one thing the trusted tool misses and add just that. Keep a fallback that still works when the smart part fails.

2. Neuralink & Synchron - Brain-Computer Interfaces

(neuralink.com, synchron.com)

Status check: Both companies' implantable BCIs for paralysis remain investigational and are being studied in clinical trials rather than being FDA-approved for general clinical use.

The barrier:

For people with severe paralysis (spinal cord injury, ALS) , conventional inputs such as hand movement, speech or eye control may be unavailable, unreliable or too limited for some tasks.

What they are:

Implants that read the brain's movement intent and turn it into computer commands, via two very different routes:

- **Neuralink:** a skull-mounted, wireless, rechargeable implant, with electrode threads placed in the brain by a surgical robot. It is in early trials, in January 2026, Neuralink reported that 21 participants were enrolled in its trials worldwide and that it had observed no serious device-related adverse events.
- **Synchron:** its Stentrode is delivered through the jugular vein and implanted in a blood vessel near the motor cortex, so no open-brain surgery is needed.

The Accessibility Angle:

Synchron's Stentrode has been demonstrated working with Apple's BCI input framework and the built-in Switch Control accessibility feature. . A participant with ALS navigated an iPad's home screen, opened apps and composed text using the Stentrode.

Why it works (HAAT):

- **Human:** severely limited movement, intact intent.
- **Activity:** messaging, browsing, controlling devices.
- **Assistive technology:** implant, wireless link, decoder.
- **Context:** home and personal-computing environments, where the user interacts with phones, tablets or computers, often with assistance from existing accessibility features.

Where it falls short:

- Invasive by nature. In Neuralink's first human trial, some electrode threads retracted after implantation, reducing the number of functional electrodes and affecting signal quality.

- Synchron's less invasive approach changes the engineering trade-offs between surgical risk, signal quality, device placement and long-term practicality.

Takeaway:

The same problem can be solved by trading precision against risk, and there's no free lunch. Also note what made Synchron's demo usable: it plugged into an existing accessibility feature instead of inventing a new ecosystem. Your project can do the same with OS accessibility tools.

3. NaviLens (navilens.com)

The barrier:

GPS can get you to a location, but it doesn't necessarily tell a blind or low-vision traveller exactly which entrance, platform, stop or sign they are facing.

What it is:

Colourful codes placed in the environment, read by a phone camera. The app detects them without GPS, Wi-Fi or Bluetooth. They target the "last few yards" problem, and are detectable from up to 100 feet away depending on code size. The physical code can remain fixed while the information associated with it, such as service or arrival information, is updated digitally. In the MTA's first pilot stage, over 5,000 codes were deployed. It can also help non-native English speakers.

Why it works (HAAT):

- Human: a blind or low-vision commuter.
- Activity: finding the exact stop, entrance or platform.
- Assistive technology: code + phone camera + audio app. The infrastructure carries the marker, while the user's phone provides the sensing and interpretation.
- Context: busy public-transit environments, including stations, platforms, buses and trains, where crowds, noise and unfamiliar layouts can make wayfinding difficult.

Where it falls short:

- Only works where someone has installed the codes.
- Needs a phone with a working camera and an app.

Takeaway:

This is "target the barrier" in action: it fixes the environment, not the person. It also separates a static physical marker from dynamic data, a neat architecture idea.

4. Tobii Dynavox (tobiidynavox.com)

The barrier:

People who can't speak or use their hands still have things to say. Conditions like ALS/MND, spinal cord injury and cerebral palsy can leave eyes as the most reliable input.

What it is:

Purpose-built speech devices controlled by gaze. The TD I-Series can be accessed by eye tracking, switch or touch, tracks eyes outdoors in bright sunlight, and has a rear-facing Partner Window for face-to-face conversation. TD Pilot adapts eye-controlled AAC and iPadOS access to an iPad-based device. The TD Talk app adds word prediction and voice banking, so users can build a synthetic voice from their own. Users can also control doors, lights and phones with their eyes.

Why it works (HAAT):

- Human: reliable gaze, limited movement and speech.
- Activity: conversation, computer use, environment control.
- Assistive technology: eye tracker, AAC software, speaker, mount.
- Context: wheelchair-mounted use, including indoor and outdoor environments, bright sunlight, and face-to-face interaction with other people.

Where it falls short:

- Access depends on funding, which varies by country and even city.

- Eye tracking requires calibration and works best for users who can reliably control their gaze, although the devices also support other access methods

Takeaway:

Design for the whole conversation, not just the user. Eyes, switch or touch means the user isn't locked out if one method stops working. The user's identity is respected by allowing them to create their own synthetic voice using voice recordings.

5. Liftware

The barrier:

Hand tremor can make eating difficult, and people may use medication, adaptive utensils and other strategies to manage it. Liftware approached the problem mechanically rather than trying to eliminate the tremor itself.

What it is:

A handle with motion sensors and a tiny computer. It detects tremor and moves the utensil opposite to it, while letting slower, intended motion like lifting through. The technique borrows active cancellation from noise-cancelling headphones. Lift Labs was founded in 2012 and acquired by Google in 2014, and now offers Liftware Steady for tremor and Liftware Level for limited upper-limb mobility.

Why it works (HAAT):

- Human: intended movement, unintended shake.
- Activity: eating.
- Assistive technology: sensors, algorithm and actuators, in a handle about the size of an electric toothbrush.
- Context: eating at home or in social settings, where the user must handle food and utensils while seated at a table.

Where it falls short:

- It reduces tremor, it doesn't eliminate it
- Evidence comes from small early studies, and it only helps with eating.
- The launch price was \$295 for a starter kit.

Takeaway:

Don't fight the body. Cancel the effort of the problem and leave the person's own movement alone.

6. Symbionic (symbionic.co)

The barrier:

Advanced prosthetic arms can be expensive, and users can also face problems with comfort, fit and ease of use. Symbionic's co-founder Rishi Krishna lost his arm in a bus accident and said his experience with existing prosthetic options motivated him to develop an alternative.

What it is:

A Chennai-based assistive-technology company founded in 2019 by Rishi Krishna and Niranjana Kumar. Symbionic develops modular upper-limb prosthetics. Its current flagship product is **Krea Adaptive**, a modular prosthetic system designed for different activities and intended to work with existing prosthetic socket systems.

Earlier reporting on Symbionic's development described custom-fitted sockets, lightweight materials and sensors designed in-house using locally available components. The same reporting described the company's 2021 smart prosthetic arm as costing ₹2 lakh at the time.

Why it works (HAAT):

- Human: an upper-limb amputee.
- Activity: everyday activities requiring an upper-limb prosthesis.

- Assistive technology: modular prosthetic arm, custom socket and activity-specific attachments.
- Context: the Indian healthcare and prosthetics environment, including the availability of fitting, customization, servicing and locally accessible prosthetic care.

Where it falls short:

- Not established that Krea Adaptive can perform every task a biological hand can perform.
- The company's current website describes Symbionic as being at the clinical stage, so its technology should not be presented as an established replacement for a biological limb.

Takeaway:

Design around the user's actual environment. Symbionic's approach combines modular hardware with customization and activity-specific attachments rather than treating a prosthetic limb as a single fixed product.

2.4 Further Reading

Standards, Frameworks & Law

- **WCAG 2.2 Quick Reference**: The W3C's customizable tool for filtering success criteria. Essential for testing your prototype against the POUR principles (Perceivable, Operable, Understandable, Robust).
- **The 7 Principles of Universal Design**: Developed by NC State University, these principles guide the creation of physical products and interfaces that require minimal adaptation for diverse users.
- **EASTIN Database (ISO 9999)**: The Global Assistive Technology Information Network database. It classifies existing assistive products using the international ISO 9999 system. Search this registry early in your ideation phase to ensure you are building to fill a gap, not cloning an existing solution.

Design Toolkits & Methodologies

- **Microsoft Inclusive Design Toolkit**: A highly readable set of manuals for digital product teams. It maps out how to design for temporary, situational, and permanent disabilities (e.g., designing for someone with one arm vs. someone holding a baby).
- **The A11Y Project**: A community-driven, open-source hub dedicated to making web accessibility easier for developers. It includes code snippets, checklists, and screen-reader testing guides.
- **Apple Accessibility Guidelines & Android Accessibility Help**: Direct documentation from the OS makers on how to hook your app into existing assistive tools like VoiceOver, Switch Control, or TalkBack (just like Synchron did in the case study).

Open Source Hardware & Developer Communities

- **Makers Making Change**: An open-source repository of 3D printable and DIY assistive technology. A goldmine for hardware hackers looking to build accessible physical interfaces, adapted switches, and mounts at a fraction of commercial costs.
- **Global Disability Innovation Hub (GDI Hub)**: Based at UCL in London, this WHO Collaborating Centre is one of the world's leading academic research and practice centres driving global disability innovation, inclusive design, and policy.
- **ATscale, the Global Partnership for Assistive Technology**: Hosted by UNOPS, this cross-sector partnership focuses on ensuring that people everywhere can access and afford life-changing assistive technology.
- **IIT Madras R2D2 (TTK Center for Rehabilitation Research and Device Development)**: Headed by Prof. Sujatha Srinivasan, this research centre is a powerhouse for context-driven assistive tech in India. They are known for deep user-centred engineering, resulting in innovations like the Arise standing wheelchair and the NeoBolt motorized add-on.

- **EnAble India**: Founded by Shanti Raghavan and Dipesh Sutariya, this pioneering NGO focuses on economic independence and dignity for persons with disabilities. They do vital work at the intersection of workplace accessibility, skill development, and systemic change in the Indian ecosystem.
- **W3C Web Accessibility Initiative (WAI)**: The consortium responsible for creating the WCAG standards you are designing to. Exploring their broader materials and history provides great insight into the philosophy behind digital accessibility.
- **Kat Holmes & Mismatch: How Inclusion Shapes Design**: Former Director of Inclusive Design at Microsoft. Her book is essential reading for hackathon teams to understand how poorly designed technology actively creates disability, and how inclusive design principles drive broader innovation.
- **Judy Heumann (judithheumann.com)**: Widely known as the "Mother of the Disability Rights Movement." While not an engineer, her decades of civil rights advocacy paved the way for the legal, structural, and policy frameworks that mandate the accessibility standards developers build to today.

Chapter 3:

TECHNICAL STACK AND TOOLING

3.1 Useful Hardware

Assistive-tech projects often start with a simple idea: sense something, figure out what it means, and help the user respond. You can do a lot with a small microcontroller, a few sensors, and a useful way of giving feedback.

Controllers

- **Arduino / ESP32** — Good for connecting sensors and controlling things like LEDs, buttons, and motors. ESP32 boards also have Wi-Fi and Bluetooth.
- **Raspberry Pi** — Useful when your project needs Python, a camera, networking, or more processing power.

Component	What you could use it for
Ultrasonic sensor	Detect nearby obstacles and measure distance
ToF sensor (e.g. VL53L0X)	Short-range distance measurement
Accelerometer + gyroscope (e.g. MPU6050)	Detect movement, orientation, gestures, or falls
Microphone	Voice input, sound detection, or sound classification
Camera	OCR, object detection, gesture recognition, or scene analysis
GPS/GNSS	Outdoor location tracking
NFC/RFID	Trigger actions using tags or identify tagged objects
Buttons / touch controls / joysticks	Alternative ways to control a device

Environmental sensors	Detect things such as light, temperature, or humidity
-----------------------	---

Giving Feedback

Sensing something is only half the project. The user needs to know what the device has detected.

- Vibration motors / haptic drivers for silent tactile feedback
- LEDs / RGB LEDs for visual feedback
- Buzzers / speakers for sounds and alerts
- Small displays when the user needs text or more detailed information

Try not to rely on colour alone. For example, instead of making red mean “danger” and green mean “safe”, combine colour with vibration, sound, text, or another clear signal.

Useful Combinations

Some simple combinations can become surprisingly useful assistive devices:

- Obstacle → distance sensor → vibration
- Sound → microphone → visual or haptic alert
- Movement → IMU → phone/computer action
- Camera → OCR/object detection → speech
- NFC tag → reader → automated action

A Few Things to Remember

- Test sensors in the environment where the device will actually be used. A sensor that behaves perfectly on a desk may behave differently outside.
- For anything safety-critical, do not rely on a single cheap sensor or an unverified AI prediction.
- Most importantly, prototype the user interaction, not just the electronics. A device can work perfectly and still be frustrating or impossible to use. Humans remain inconveniently important to the design.

3.2 Accessible Design Toolkits

Every interactive element has hidden requirements. A dropdown has to open with the keyboard, trap focus properly, announce itself to a screen reader, and close on Escape. Getting that right from scratch takes days. Accessible design toolkits are libraries where someone has already implemented many of the difficult interaction and accessibility patterns and tested them across common assistive-technology scenarios, so you get those foundations without building everything from scratch.

Component Libraries (Web)

1. **shadcn/ui** (*React*)

The quickest route to something that looks good. It gives you components you add directly to your own repo, with Tailwind-based styling and primitives from Base UI, Radix or React Aria. New projects now default to Base UI, while Radix remains supported. Pick it if you know React and want speed. Watch out: once the component code is in your repo, it's yours, so re-check contrast, focus and interaction behaviour if you restyle or modify anything.

2. **React Aria Components** *(React)*

A component library focused heavily on accessibility and internationalisation, with components designed around keyboard interaction, screen-reader semantics and complex interaction patterns. Pick it when those behaviours are a major requirement of your project.

3. **Base UI** *(React)*

Unstyled, accessible building blocks such as dialogs, menus, selects and tabs. It bundles no CSS, so you control the look completely. It comes from the creators of Radix, Floating UI and Material UI. Pick it when you want accessible interaction behaviour without adopting a predefined visual design.

4. **Headless UI** *(React, Vue)*

The simplest option for Tailwind projects, with a smaller component set, so check it has what you need.

5. **Radix UI** *(React)*

A mature collection of accessible, unstyled React primitives. It is still supported by shadcn/ui, although it now defaults to Base UI for new projects.

6. **Ark UI** *(React, Solid, Vue, Svelte)*

A headless library of 45+ accessible components with the same API across React, Solid, Vue and Svelte, with accessibility behaviour and WAI-ARIA patterns built into its components. It's particularly useful for teams working outside React because it provides the same component library across React, Solid, Vue and Svelte.

7. **Ariakit** *(React)*

A lightweight accessible toolkit, worth a look if the bigger libraries feel heavy.

8. **GOV.UK Frontend** and **USWDS** *(the US Web Design System)*

provide reusable styles, components and patterns that can be used across different web stacks. They were built for public services, so accessibility is a core design requirement and every component is designed around forms, errors and clear language. They're a strong fit for anything involving public information or form-heavy flows.

Mobile

Each platform has accessibility support built in:

- **Android:** Jetpack Compose provides semantics for exposing UI information to accessibility services. Accessibility Scanner is a separate testing tool that can identify some common accessibility problems.
- **iOS:** SwiftUI's accessibility modifiers and Xcode's Accessibility Inspector.
- **Flutter:** the *Semantics* widget.
- **React Native:** accessibility props such as *accessibilityLabel* and *accessibilityRole*.

Type and Colour

Atkinson Hyperlegible Next is a free typeface from Braille Institute designed with legibility in mind, including distinctions between easily confused characters such as B/8 and 1/l/I. It has seven weights and glyphs supporting over 150 languages. Braille Institute also provides a separate *Atkinson Hyperlegible Mono* typeface for coding and tabular text. Check script coverage before using it for Malayalam; Noto's Malayalam fonts are a safer fallback.

For colour, use WebAIM's Contrast Checker or the contrast tools in your browser's DevTools. For normal-sized body text, aim for at least **4.5:1** contrast. Large text has a lower minimum under WCAG. Chrome DevTools can also emulate colour-blindness and other vision deficiencies, so you can check your design in seconds. If you design in Figma, a plugin like Stark can bring contrast checks into your mockups.

Check Your Work

- **axe DevTools:** a browser extension that scans a page and lists issues with suggested fixes.
- **Lighthouse:** built into Chrome DevTools, it runs an automated accessibility audit and gives you an accessibility score.
- **WAVE:** a tool from WebAIM that marks problems directly on the page.
- **Screen readers:** NVDA (free, Windows), VoiceOver (built into Apple devices) and TalkBack (Android). Test the main flow with a screen reader rather than trying to simulate blindness by closing your eyes.

Automated scans only catch part of the accessibility problems. Always add a keyboard-only pass and a screen reader pass on your core flow.

What libraries won't do for you

A toolkit handles how components *behave*. Everything about your content is still on you:

- Writing useful **alt text and labels**
- Choosing **colours with enough contrast**
- Putting things in a sensible **reading order**
- Using **plain language** and helpful error messages
- Making sure the **core idea** works for your user
- Making sure the interface still works when users **zoom in or increase text size**

Habits that save you

- Native HTML first. Use `<button>`, `<label>` and `<nav>` before reaching for ARIA, and add ARIA only when native elements can't do the job.
- Never remove focus outlines without replacing them.
- Size text in *rem* so it scales when users enlarge it.

- Test with just your keyboard after wiring in any library. Your own styles can override good defaults.
- Do a dry run before the event. Install your chosen library, build one screen, and run it through a keyboard-only pass. You don't want to be debugging setup at hour two.

3.3 Specialized Accessibility APIs

Most accessibility projects come down to translating one kind of input into another: speech into text, a scene into a description, a gesture into a command. These APIs and models do the heavy lifting, so you don't have to train anything yourself.

Four questions to ask before picking any API:

- **Does it need the internet?** If your user loses signal, does the core function die?
- **How fast does it respond?** A delay of a second is fine for a caption, but not for an obstacle warning.
- **Does it handle your user's language and accent?** Test with real Malayalam speech, not just English.
- **Where does your user's data go?** Voices and faces sent to a cloud service are personal data.

Built into the Browser or Phone

These are free, need no sign-up, and are the quickest route to a working demo.

1. **Web Speech API.** Lets a web page speak text aloud (speech synthesis) and, in some browsers, listen (speech recognition). Speech recognition is often processed by an online service, although some supporting browsers can use on-device recognition. Check the browser's implementation before assuming speech stays on the device.

2. **ARIA live regions.** An HTML feature that tells screen readers to announce changes as they happen, such as a new message, a status update or an error. If your app updates the screen without the user doing anything, they won't know unless you use this.
3. **Vibration API.** Makes a phone vibrate from a web page. Browser support is limited, including on iPhones, so build the same signal through sound or a visual cue too.
4. **Web Bluetooth and Web Serial.** Let a web page talk directly to your ESP32 or Arduino, which is a very quick way to build a hardware + software project with one codebase. Browser support is limited. Chrome and Edge support these APIs on supported platforms, but Safari and Firefox generally do not, so test the exact browser and device you plan to use.
5. **Android AccessibilityService.** Lets an app read what's on screen and act on it, which is how screen readers and switch controls work. It's powerful, and users have to enable it manually.

Hearing, Speech and Indian Languages

1. **Sound recognition:**
MediaPipe's Audio Classifier can run an audio-classification model on-device. With an appropriate pretrained model, this can be used to recognise sounds such as alarms, doorbells or sirens. Another option is YAMNet, a pretrained model that recognises hundreds of everyday sound categories. Together with a vibration or visual alert, these are building blocks for sound-awareness tools.
2. **Bhashini:**
India's national language platform, open to developers. It exposes ASR (speech recognition), neural machine translation and text-to-speech for Indian languages through APIs, with models from institutes like IITB, IITM and CDAC. You use a pipeline to chain them, for example speech in, translation, speech out. It covers translation across India's 22 scheduled languages.

3. **Sarvam:**

Speech-to-text built for Indian languages, with Malayalam and Indian English among its supported languages. Its realtime streaming API supports 24 language codes, including automatic language detection, and can handle code-mixed speech.

4. **AI4Bharat IndicTrans2 and Indic-TTS:**

IndicTrans2 is an open-source translation model supporting all 22 scheduled Indic languages, including Malayalam. Indic-TTS provides open-source text-to-speech models for 13 Indian languages, including Malayalam. Good when you want to run models yourself rather than depend entirely on a commercial cloud API.

Vision and Gesture

1. **MediaPipe Task:**

Google's on-device ML toolkit, running on web, mobile, desktop and IoT. It includes gesture recognition, hand landmark detection, image classification and object detection. Its Face Landmarker tracks facial landmarks and can output blendshape scores representing facial movements such as eye and mouth movements, which makes it useful for alternative controls. Because it runs on the device, it works without the cloud and keeps camera data local.

2. **Project Gameface:**

Google's open-source hands-free mouse uses head movement and facial gestures to control a cursor. It was co-designed with Lance Carr, a gamer with muscular dystrophy, and has Windows and Android implementations. It uses MediaPipe and includes a gesture size setting that lets users adjust how large a gesture must be before it triggers an action. The original GitHub repository was archived in September 2025, so treat it as a reference or starting point rather than an actively maintained project.

3. **OCR (reading text from images):**

Tesseract is a widely used open-source OCR engine. Cloud vision

services offer OCR too. OCR accuracy depends heavily on image quality, text layout, handwriting and language/script support. Always test with your own language, and give users a way to correct a wrong reading.

Language, Symbols and Braille

1. **ARASAAC:**

A large free library of pictograms for AAC (augmentative and alternative communication), with a public API. Useful for picture-based communication boards and for users who find text hard. Check the license terms before using it beyond the hackathon.

2. **Liblouis:**

An open-source braille translation library, useful if your hardware drives a braille display or embosser. Check that it has a table for the language and braille code you need.

Maps, Places and Routes

1. **Google Places API.** Place records can include accessibility details: wheelchair-accessible entrance, parking, restroom and seating. These fields may be unavailable, so design for missing data and say "unknown" instead of "not accessible."
2. **OpenStreetMap + Overpass API.** OSM contributors tag places and paths for wheelchair access (values like yes, limited and no), and you can query that data for free. It's also a project opportunity: if your area is poorly mapped, the gap is the finding.
3. **OpenRouteService.** Routing with a dedicated wheelchair profile that uses OSM information and wheelchair-specific restrictions such as surface type, smoothness, kerb height and maximum incline. You sign up for an API key. Check coverage for your area before building on it.

Reality Checks

- **Sign language is not a solved problem.** It is a full language with its own grammar and use of facial expression, body movement and space, not just hand shapes. A prototype that recognises a limited set of gestures should not be presented as a general sign-language translator. Be explicit about exactly which signs, users and conditions your prototype supports.
- **Every ML output can be wrong.** For anything a user acts on (a street name, a medicine label, a warning), give users a way to confirm important outputs, and avoid presenting model confidence as a guarantee of correctness.
- **Free tiers change.** Check current limits and set up accounts and keys before the event. Links to pricing and docs are in Further Reading.

Quick Reference

I need to...	Reach for...
Speak text aloud in a browser	Web Speech API
Announce changes to a screen reader	ARIA live regions
Recognise sounds	MediaPipe Audio Classifier, YAMNet
Control a device with face or head	MediaPipe Face Landmarker, Project Gameface
Track hands or recognise gestures	MediaPipe Hand Landmarker, Gesture Recognizer
Work in Malayalam and other Indian languages	Bhashini, Sarvam, IndicTrans2, Indic-TTS
Read text from images	Tesseract, cloud OCR

Make picture-based communication	ARASAAC
Build wheelchair-aware routing	OpenRouteService, OSM data
Look up recorded accessibility information	Google Places API, OSM tags
Connect a web page to hardware	Web Bluetooth or Web Serial

3.4 General Hackathon Toolkit

These are the tools most teams can use regardless of what they are building: general-purpose AI models, speech, model hosting, databases, deployment and the bits that get a prototype in front of a judge. Free tiers and quotas change, so create accounts and test your actual workflow before the event.

General-purpose AI

1. **OpenRouter:** One API in front of many models, so you can switch models by changing the model ID. Its free tier currently lists 25+ free models, and free-model limits are currently 50 requests per day, with higher limits after a qualifying credit purchase. Which models are free can change over time.
Watch out: a free quota disappears quickly when you're repeatedly testing prompts. Cache responses while developing.
2. **Google Gemini API:** Google's API for Gemini models, with a free tier that provides access to selected models within model-specific limits. Rate limits are measured per project and can include requests per minute, tokens per minute and requests per day. Check the limits for the exact model you are using. Don't assume that a free tier means unlimited access.
3. **Groq:** An API for fast inference on supported open models. Its limits vary by model and include requests, tokens and, for audio models, audio seconds. For example, the current free limits list 30

requests per minute for several text models and 20 requests per minute for Whisper models.

Pick it when **latency matters** and the models it offers fit your task.

4. **Hugging Face:** A large catalogue of open models, datasets and Spaces. It is useful when you need a specialised model rather than a general chatbot, and Spaces can host interactive demos built with tools such as Gradio or Streamlit.
5. **Replicate:** A hosted API for running machine-learning models without setting up the model infrastructure yourself. It hosts thousands of public models and maintains a collection of official models with stable APIs and predictable pricing.
Useful when you need something more specialised, such as image generation, vision or audio, and don't want to run the model locally.
6. **Run it locally:** Tools such as **Ollama** let you run supported models on your own computer (low parameter models like Gemma/Qwen can also be fine-tuned and run on the edge) instead of sending every prompt to a cloud API. That can be useful for offline operation and for keeping input data on the machine.

Watch out: local inference needs enough RAM, storage and compute for the model you choose.

Speech

1. **Whisper:** OpenAI's open-source automatic speech-recognition model family. The official release includes multilingual models in several sizes, and the code is MIT-licensed. You can run the models yourself or use hosted implementations.

Whisper is designed around chunks of audio rather than a true streaming microphone pipeline, so a live application needs to handle chunking and latency itself or use a streaming implementation.

- Larger models need substantially more compute and memory. Test the model on the actual hardware you will use.
 - Accuracy varies by language, recording conditions and speaker. Test with your users' actual speech rather than assuming English results generalise.
2. **ElevenLabs.** Provides text-to-speech and speech-to-text through APIs. Its current free plan includes 10,000 credits per month, with credit costs depending on the model and product. Its Scribe speech-to-text system supports 90+ languages, including Malayalam. Language support differs between speech-to-text and text-to-speech models, so check the exact model before building around a language. Free-plan terms and API limits can change.

Refer to 3.3 for Indian-language speech tools

Build and Ship

1. **GitHub Pages.** A simple option for static HTML, CSS and JavaScript sites directly from a GitHub repository. It is available with GitHub Free for public repositories.
2. **Vercel.** Connect a Git repository and get automatic deployments and preview URLs for changes. It is particularly convenient for modern web applications.
3. **Cloudflare Pages.** Deploy static sites from GitHub or GitLab, with automatic deployments and preview deployments. It can also run server-side code through Pages Functions.
4. **Firebase.** Firebase Hosting handles web deployment, while Firebase also provides services such as Authentication and Cloud Firestore. Hosting can serve static sites and can be paired with backend services for dynamic applications.
5. **Supabase.** A managed Postgres database with authentication, storage and other backend services. Its Auth system supports

methods including passwords, magic links and one-time passwords. The free plan currently allows two free projects.

6. **Streamlit or Gradio.** Fast ways to put a Python or ML idea behind a usable interface. Both can also be deployed through Hugging Face Spaces.
7. **Modal.** A serverless platform for running compute-heavy Python and ML workloads without maintaining your own server. Its Starter plan currently includes \$30 of compute per month, although usage beyond the included amount is billed.
Useful when a model is too heavy for your laptop but you only need compute intermittently.
8. **Cloudflare Tunnel.** Publishes a local application through a public hostname, which is useful when a phone or judge needs to reach a server running on your laptop.

Don't choose a tool just because it is impressive. Choose it because it removes a piece of work you don't want to spend the event doing.

Chapter 4:

PROTOCOLS

4.1 Rules and Guidelines

General Rules

- Maintain discipline and professionalism throughout the event.
- Keep your workspace clean and organised.
- Respect other teams, mentors, judges, volunteers, and organisers.
- Do not interfere with another team's work or equipment.
- Participants are not allowed to contact their mentors during the Hackathon.
- Organisers are not responsible for lost or damaged personal belongings.
- Follow all venue and safety instructions.

Violating these rules may result in disqualification.

Important

- Choose only one problem statement.
- Your team has complete liberty on how you choose to solve the problem, but the solution must directly address the selected challenge.
- Manage your time carefully. No additional development time will be provided after the deadline.
- Only an idea pitch or proposal will not be accepted as a project
- A working prototype is encouraged over an unfinished, overly complex product.
- Teams should be prepared to explain their design and technical decisions.

The **hackathon begins at 8:00 AM** and **ends at 3:00 PM**. Teams are required to ideate, design and build their projects within this 7 hour window. Any presentations, pitches, etc. must also be developed in this time span. Project Overview Document must also be completed before hackathon time ends.

4.2 Components and Tools

For building your innovative ideas, we will be providing a complimentary kit of components:

- ESP32 + WIRES
- ULTRASONIC SENSOR
- JUMPER WIRES
- LED - 3
- RESISTORS - 3
- 9V BATTERY
- BREADBOARD - SMALL

Apart from these, there will be a **Components Desk** from which teams can **borrow** additional components. These components **must be returned** after the event.

Component Desk - Rules

Availability of components from the Component Desk will be limited. All teams will be provided with a Component Request Form which they can pass onto their assigned usher, who will then deliver the components to their table. **Component request window is between 8:30 AM and 10:30 AM**

- Specialised components should be brought by the team where possible.
- Shared components will be distributed on a first-come, first-served basis.
- Equipment such as soldering irons or glue guns may only be used at designated stations.
- Return borrowed components before leaving the venue.

Components Desk - Catalog

- ESP32
- ARDUINO UNO
- RAIN DROP SENSOR
- HUMIDITY/TEMP SENSOR WITH PCB DHT11
- SOIL MOISTURE SENSOR
- SERVO MOTOR
- MQ135 GAS SENSOR MODULE ADIY
- BUZZER PCB MOUNT
- ECG SENSOR MODULE
- HEART RATE SENSOR MODULE MAX30102
- 6-AXIS GYRO MODULE MPU6050
- ECG SENSOR
- BREADBOARD
- DHT
- BATTERY HOLDER
- ACTIVE BUZZER
- VIBRATION SENSOR
- MOTOR DRIVER BOARD
- STEPPER MOTOR
- GAS SENSOR
- MQ3
- MQ8
- MQ5
- RFID
- RESISTORS, TRANSISTORS
- GPS
- LED
- BATTERIES

4.3 AI and External Resources

Teams may use AI tools and external resources as development aid.
However:

- Teams must understand the work they submit.
- Do not present an entirely pre-built project as work created during the hackathon.
- Explain models, datasets, APIs, or other resources used during your pitch
- Respect the licenses and usage terms of external resources.

4.4 Originality and Fair Play

- Projects must be developed by the participating team.
- Plagiarism and copying another team's work are prohibited.
- Existing tools and technologies may be used, but your team's contribution must be clear.
- Teams must not intentionally misrepresent their work.

Any attempt to gain an unfair advantage through:

- Interfering with another team
- Misusing event resources
- Disrupting the network
- Circumventing event systems
- Getting outside help during the hackathon

Will result in immediate disqualification

4.5 Judging Rubric - Phase I

Criteria	Explanation	Weight
Concept & Vision	Idea originality and a realistic roadmap to scale it	35%
Theme Alignment	Real-world impact solved from the user's perspective, not assumptions	35%
Technical Execution	Working code or hardware that you can defend in technical Q&A	30%

4.6 Venue Details

Location - The School of The Good Shepherd, Akkulam

Rooms - Sr. Computer Lab, Library

Date: 10th October, 2026

Registrations: 7:00 AM - 8 : 00 AM

Hackathon: 8:00 AM to 3:00 PM

Available

- Team workspaces
- Wi-Fi
 - **Password:** hacktvm@26
- Power outlets
- Unlimited snacks & refreshments
- Lunch

- Restrooms
- First-aid facilities

Bring

- Laptop + charger
- Extension board/power strip
- Required hardware
- Necessary adapters
- Student ID (Compulsory)

Safety

- Use electrical tools only in designated areas.
- Report damaged equipment immediately.
- Follow all instructions from organisers and venue staff